MLIR Tasks – Report

1 Part 1: Matrix Multiplication in MLIR

1.1 Task 1: Build LLVM/MLIR and the Starter Project

1.1 Clone LLVM/MLIR

The following commands clone LLVM at the required tag and create the workspace:

```
mkdir -p ~/mlir-work
cd ~/mlir-work

git clone --branch llvmorg-23.1.3 --depth 1 \
  https://github.com/llvm/llvm-project.git llvm-project

cd ~/mlir-work/llvm-project
git describe --tags
```

1.2 Configure LLVM/MLIR

```
cmake -G Ninja \
  -S ~/mlir-work/llvm-project/llvm \
  -B ~/mlir-work/build \
  -DCMAKE_BUILD_TYPE=Release \
  -DCMAKE_INSTALL_PREFIX="$HOME/mlir-work/llvm-install" \
  -DLLVM_ENABLE_PROJECTS=mlir \
  -DLLVM_TARGETS_TO_BUILD=host \
  -DLLVM_ENABLE_ASSERTIONS=ON \
  -DLLVM_INSTALL_UTILS=ON
```

1.3 Build and install LLVM/MLIR

```
cmake --build ~/mlir-work/build --target install --parallel 2
```

```
~/mlir-work/llvm-install/bin/mlir-opt --version
```

The reported version was LLVM 23.1.3.

1.4 Clone and build the starter project

```
cd ~/mlir-work

git clone https://github.com/h4midf/MLIR-task-tud.git
```

```
cmake -G Ninja \
  -S ~/mlir-work/MLIR-task-tud \
  -B ~/mlir-work/starter-build \
  -DMLIR_DIR="$HOME/mlir-work/llvm-install/lib/cmake/mlir" \
  -DLLVM_EXTERNAL_LIT="$HOME/mlir-work/build/bin/llvm-lit"
```

```
cmake --build ~/mlir-work/starter-build --parallel 2
```

```
cmake --build ~/mlir-work/starter-build \
  --target check-standalone-opt --parallel 2
```

1.2 Task 2: Express Matrix Multiplication in Linalg

1.2.1 Named Matrix Multiplication

```
func.func @matmul_named(
      %A: memref<?x?xf64>,
      %B: memref<?x?xf64>,
      %C: memref<?x?xf64>) {
    linalg.matmul
      ins(%A, %B :
          memref<?x?xf64>, memref<?x?xf64>)
      outs(%C : memref<?x?xf64>)
    return
  }
```

1.2.2 Generic Matrix Multiplication

```
 func.func @matmul_generic(
      %A: memref<?x?xf64>,
      %B: memref<?x?xf64>,
      %C: memref<?x?xf64>) {
    linalg.generic {
      indexing_maps = [
        affine_map<(d0, d1, d2) -> (d0, d2)>,
        affine_map<(d0, d1, d2) -> (d2, d1)>,
        affine_map<(d0, d1, d2) -> (d0, d1)>
      ],
      iterator_types = [
        "parallel", "parallel", "reduction"
      ]
    }
    ins(%A, %B :
        memref<?x?xf64>, memref<?x?xf64>)
    outs(%C : memref<?x?xf64>) {
    ^bb0(%a: f64, %b: f64, %c: f64):
      %product = arith.mulf %a, %b : f64
      %sum = arith.addf %c, %product : f64
      linalg.yield %sum : f64
    }
    return
  }
```

1.2.3 Generic Versions

```
~/mlir-work/llvm-install/bin/mlir-opt ~/mlir-work/MLIR-task-tud/matmul.mlir --linalg-
    generalize-named-ops -o ~/mlir-work/MLIR-task-tud/matmul-generalized.mlir
```

Both became:

```
#map = affine_map<(d0, d1, d2) -> (d0, d2)>
#map1 = affine_map<(d0, d1, d2) -> (d2, d1)>
#map2 = affine_map<(d0, d1, d2) -> (d0, d1)>

...

  func.func @matmul_xxxxxx(%arg0: memref<?x?xf64>, %arg1: memref<?x?xf64>, %arg2:
    memref<?x?xf64>) {
    linalg.generic {indexing_maps = [#map, #map1, #map2], iterator_types = ["parallel
    ", "parallel", "reduction"]} ins(%arg0, %arg1 : memref<?x?xf64>, memref<?x?xf64>)
    outs(%arg2 : memref<?x?xf64>) {
    ^bb0(%in: f64, %in_0: f64, %out: f64):
      %0 = arith.mulf %in, %in_0 : f64
      %1 = arith.addf %out, %0 : f64
      linalg.yield %1 : f64
    }
    return
  }
```

I suspect this is because both implementations perform the same computation.

1.2.4 Questions

Q1 Indexing map and iterator types can both be seen in the generic version and the upstream passes. At iteration (d_0, d_1, d_2), it accesses $A[d_0, d_2]$,$B[d_2, d_1]$,$C[d_0, d_1]$

Q2 C is an output operand because the operation writes its result there. The block argument holds the initial value of the accessed matrix element before being incremented by the value of AB. `linalg.matmul` was not initialized to zero.

Q3 d_2 is the reduction iterator. This is because a single value of $C[d_0, d_1]$ depend on many values of d_2. The verification succeeded after changing it to parallel but it is not a correct description of the intended computation.

1.3 Task 3: Lower to loops

Q4 The following passes were used:

```
~/mlir-work/llvm-install/bin/mlir-opt matmul.mlir \
  --convert-linalg-to-loops \
  -o matmul-scf.mlir
```

```
~/mlir-work/llvm-install/bin/mlir-opt matmul.mlir \
  --convert-linalg-to-affine-loops \
  -o matmul-affine.mlir
```

I found the answer by using the help command and filtering for loops

```
~/mlir-work/llvm-install/bin/mlir-opt --help-hidden | grep -A 8 -i linalg-interchange
```

1.4 Task 4: Lower to loops

Q5 The order I chose was to do (d_0, d_2, d_1). This would make it so that the row jumps are accessed contiguously and a single element in C would be complete after the completion of the second inner loop.

Q6 I was unable to find a command in my project that would do this automatically, so I generated affine loops and manually changed the order. The same approach worked on the named version.

Q7 Moving the reduction loop is legal because each element of C still gets updated using the same values of A and B. The interchange only changes the order in which we visit the elements of C. For each element of C, the accumulation order over k stays the same, so the result does not change.

1.5 Task 5 Lower to LLVM, link and run

Q8 I lowered the original Linalg operations into affine loops, then converted the loops into control flow and the arithmetic, memory, and functions into the LLVM dialect. These steps must happen in the correct order because each stage prepares the program for the next. The LLVM dialect was translated into LLVM IR, compiled into object files, and linked with my C driver to create executables. Both executables ran successfully and produced the expected matrix product.

2 Part 2: A Stencil Dialect

2.1 Task 2 Lower Jacobi-2D to Linalg

Q9 Parallel iterators are valid because ‘stencil.apply‘ is an out-of-place operation. Each output element is computed independently using values from the input memref, and no iteration modifies the input. Therefore, iterations do not depend on one another, and they can safely execute in parallel. The output elements are also written to distinct memory locations, so there are no write conflicts between iterations.

2.2 Task 3 Lower Seidel-2D to loops

Q10 Unlike `stencil.apply`, `stencil.sweep` is an in-place operation. Each iteration reads and updates the same memref, meaning later iterations may depend on values written by earlier ones. Parallel execution could change the order of these updates and therefore produce incorrect results. Sequential nested loops are necessary to preserve the required lexicographic traversal.

2.3 Task 4 Compile and run

Q11 Yes. This is because both implementations use the same floating-point precision (double), perform the arithmetic operations in the same order, and use the same initialization values. Therefore, the calculations produce the same rounded floating-point results.

Q12 Not in all cases. In the case of multiplying by the reciprocal, two values have to be rounded. The first is the value of the reciprocal itself. The second is the value of the multiplication. In the division case only one rounding occurs. LLVM’s arcp fast-math flag allows division to be replaced by multiplication with an approximate reciprocal. The broader fast flag permits additional transformations that may disregard strict IEEE floating-point behavior.